

tp2

Exercice 1

1.1

A l'aide de commandes, créez l'arborescence donnée ci-dessous et déplacez-vous à l'intérieur

```
cd $HOME
mkdir -p ~/ {cours,TP}/
touch README LISEZMOI ~/cours/notes1 ~/cours/notes2
```

Notes :

L'argument `-p` permet de créer les chemins par lien de parentés et donc d'avoir la possibilité de créer des multiples enfants si nécessaire ou plusieurs dossiers parents dans le même dossier racine.

La mise en place des `{ }` dans la commande permet de créer 2 arguments qui seront pris en charge lors de création pour créer automatiquement 2 dossiers.

1.2

Par défaut, quels sont les droits d'accès des fichiers créés ? Et répertoires créés ?

```
ls -l | grep cours
drwxrwxr-x - pazema 29 Sep 14:07 cours
ls -l | grep TP
drwxrwxr-x - pazema 29 Sep 14:06 TP
```

Les dossiers et les droits sont définies par `umask` permettant de définir par défaut lors de la création d'un fichier ou d'un dossier les droits défini.

Ici la commande `umask` nous met en place `002` ce qui permet au système de dégager la partie `o+w` et donc d'empêcher l'écriture dans le dossier pour les autres utilisateurs.

1.3

Rajouter le droit d'écriture pour le tous au fichier notes1

```
chmod a+w cours/notes1
```

La commande `chmod`, dans le cas présent, permet de donner le droits d'exécution pour tout l'utilisateur actif, les utilisateurs présent dans le même groupes que l'utilisateur actif, et les autres utilisateurs ne faisant pas partie du groupe de l'utilisateur actif.

1.4

Modifier les droits d'accès du fichier LISEZMOI pour qu'ils soient à 521. Vérifiez par une commande.

```
chmod 521 LISEZMOI
```

Vérification des droits mis en place par la commande `chmod` via `stat` qui va nous permettre de voir et de remarquer la configuration des droits en `r-x-w---x`, qui, en description total :

r-x : l'utilisateur à le droit de lecture et d'exécution mais pas d'écriture -w- : le groupe de l'utilisateur actif ont les droits d'écriture -x : les autres utilisateurs ont le droit d'exécution.

```
stat LIZEZMOI
  File: LIZEZMOI
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 252,1    Inode: 8663702      Links: 1
Access: (0521/-r-x-w---x)  Uid: ( 1000/  pazema)   Gid: ( 1000/  pazema)
Access: 2025-09-29 14:10:05.359468198 +0200
Modify: 2025-09-29 14:07:05.040143494 +0200
Change: 2025-09-29 14:10:15.158124574 +0200
Birth: 2025-09-29 14:07:05.040143494 +0200
```

1.5

Supprimer ensuite tous les répertoires et fichiers créés

```
rm -fr cours/ TP/ README LIZEZMOI
```

La commande `rm` avec les arguments `-fr` pour `--force` et `--recursive` permet de supprimer les fichiers présents dans les dossiers où ils ne sont pas vide pour permettre de tout supprimer.

La partie `-f` permet de détruire sans demander confirmation.

Le schéma de destruction est défini comme ceci

- suppression cours/notes1
- suppression cours cours/notes2
- suppression cours/
- suppression TP/
- suppression README
- suppression LIZEZMOI

2

2.1

Dans votre home directory, créer un répertoire essai.

```
mkdir $HOME/essai
```

2.2

Copier les fichiers `/etc/passwd` et `/etc/group` dans le répertoire essai sous des noms différents.

```
cp /etc/passwd $HOME/essai/password
cp /etc/group $HOME/essai/groupes
```

2.3

Créer dans essai un répertoire copies.

```
mkdir $HOME/essai/copies
```

2.4

Déplacer un des fichiers de essai dans copies.

```
mv $HOME/essai/password $HOME/essai/copies/
```

2.5

Créer un répertoire titi dans copies.

```
mkdir $HOME/essai/copies/titi
```

2.6

Supprimer le droit d'exécution 'x' pour le répertoire copies.

```
chmod a-x $HOME/essai/copies/
```

2.7

Taper ls copies. Que remarquez-vous?

```
ls $HOME/essai/copies/  
d----- - root - titi
```

la commande indique que le dossier titi n'as plus aucun droit et indique bien que l'utilisateur n'as plus le droit de modification dans les dossiers enfants.

2.8

Détruire le contenu du répertoire copies avec la commande rm. Que remarquez-vous?

```
rm copies/*  
rm: cannot remove 'copies/titi': Permission denied
```

La suppression du droit d'exécution empêche `rm` d'être lancé dans dossier copies/ pour supprimer les dossiers ou fichiers existant en son sein.

2.9

Ajoutez le droit d'exécution 'x' pour le répertoire copies.

```
chmod a+x $HOME/essai/copies/
```

2.10

Chercher à l'aide de man l'option de la commande rm permettant de détruire le répertoire copies

```
rm -r $HOME/essai/copies/
```

3

3.1

Afficher le contenu du fichier `/usr/include/dialog.h` avec la commande `cat`.

```
cat /usr/include/dialog.h
```

3.2

Faire `cat` sans nom de fichier. Que remarquez-vous? Sortir avec CTR-D.

```
cat
```

Je remarque que la commande `cat` attend une entrée standard pour afficher son contenu.

3.3

Faire `cat /etc/group`.

```
cat /etc/group
```

3.4

Afficher le même fichier avec la commande `more`.

```
more /etc/group
```

La commande `more` permet de lire un fichier ligne par ligne.

3.5

Faire `whatis ls`. Que remarquez-vous? De même avec `whereis` et `which`.

```
whatis ls
ls (1) - list directory contents
whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
which ls
/usr/bin/ls
```

Je remarque que la commande `whatis` permet d'afficher la description d'une commande. Je remarque que la commande `whereis` permet de trouver le chemin d'une commande. Je remarque que la commande `which` permet de trouver le chemin d'une commande.

4

4.1

On se propose de tester la commande `ln`. Pour cela :

Créer un fichier de test nommé `original` et un lien physique sur ce fichier nommé `physique`.

Ecrivez à l'aide de `nano` du texte (une 20e de caractères) dans le fichier `original`.

```
touch original
ln original physique
nano original
```

4.2

Ouvrir les fichiers original et physique. Que constate-t-on après édition du fichier physique ?

On constate que le contenu du fichier physique est identique au contenu du fichier original.

4.3

Créer un lien symbolique sur ce fichier nommé symbolique.

```
ln -s original symbolique
```

4.4

Faites un `ls -l`. Que constatez-vous ?

Je constate que le fichier original est un fichier ordinaire et que le fichier physique est un lien physique.

4.5

Modifier le contenu du fichier original. Que constate-t-on au niveau du fichier symbolique ? Et au niveau du fichier physique ?

```
echo "Hello World" > original
```

Je constate que le contenu du fichier symbolique est identique au contenu du fichier original. Le fichier physique n'a pas été modifié et le texte ancien est encore présent.

4.6

Effacer le fichier original puis ouvrir le fichier symbolique. Que se passe-t-il ?

```
rm original
nano symbolique
cat symbolique
[bat error]: 'symbolique': No such file or directory (os error 2)
```

le fichier symbolique n'est plus accessible et ne permet plus d'accéder au contenu du fichier original car le symbole est cassé.

4.7

Ouvrez le fichier physique. Que se passe-t-il ? Concluez

```
cat physique
je suis un texte long
```

Je conclus que le fichier physique est toujours accessible et contient le contenu du fichier original avant d'avoir été altéré par l'utilisateur.

5 La commande grep

Effectuez les recherches suivantes sur ce dictionnaire à l'aide du “filtre” `grep` sur le fichier `dico_francais.txt`

1

Liste des mots se terminant par les lettres `cot`

```
grep 'cot$' dico_francais.txt
```

2

Commençant par ab et terminant par t

```
grep '^ab.*t$' dico_francais.txt
```

3

commençant par `[a-l]`

```
grep '^([a-l])' dico_francais.txt
```

4

compter le nombre de mots commençant par `v`

```
grep '^v' dico_francais.txt | wc -l
```